Insurance AI Lab Guide

Module 3: Vector Search & Retrieval

FAISS Similarity Search | Semantic Search with Metadata Filters
Hybrid Keyword + Embedding Search | Latency & Accuracy Benchmarking

AWS EC2 Ubuntu | Code-Server IDE | Python 3.10+ | OpenAI API | LangChain

Real-World Use Case

An insurance operations team receives thousands of daily customer queries: "Does my policy cover knee surgery?", "I need to file a flood claim for my car", "What is the NCB on my motor policy?". You will build a complete vector-powered retrieval pipeline that instantly finds the most relevant policy clauses, with metadata filtering by policy type and claim type, and a hybrid engine combining keyword precision with semantic understanding.

Lab	Topic	What You Build	Key Stack	Time
Lab 9	**FAISS Vector Store**	Build, persist, load FAISS; k-NN search with scores	*faiss-cpu, OpenAIEmbeddings*	40 min
Lab 10	**Semantic + Metadata Filters**	Filter by policy_type, claim_type, section before ranking	*LangChain FAISS filter=*	45 min
Lab 11	**Hybrid Search (BM25 + FAISS)**	BM25 + FAISS ranked lists fused with RRF	*rank-bm25, FAISS, RRF*	50 min
Lab 12	**Retrieval Benchmarking**	Latency, Precision@3 and Recall@3 across all 3 engines	*time, numpy, json*	40 min

Prerequisites	Install All Packages First

Modules 1 and 2 completed (Labs 1-8) AWS EC2 Ubuntu running, Code-Server open OpenAI API key set in .env file Virtual environment activated	`pip install faiss-cpu rank-bm25` `pip install langchain langchain-openai pip install langchain-community tiktoken pip install numpy`

Table of Contents

Shared Insurance Knowledge Base

30 documents across Health, Motor and Life — used in all 4 labs

Why a Shared Dataset?

All four labs use the identical 30-document corpus so results are directly comparable across FAISS, semantic filter search, hybrid search, and benchmarking. Each document carries four metadata fields: policy_type, claim_type, section, doc_id.

Setup

```
cd ~/insurance_ai_lab
mkdir -p lab9 lab10 lab11 lab12 shared
touch shared/corpus.py
code shared/corpus.py
```

shared/corpus.py

```
# shared/corpus.py
from langchain_core.documents import Document

CORPUS = [
# ■■ Health Insurance (H001-H010) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Document(page_content=
"Pre-existing diseases like diabetes and hypertension are excluded for 36 months "
"from first policy inception. Conditions diagnosed within 48 months before policy "
"start are considered pre-existing.",
metadata={"policy_type":"health","claim_type":"exclusion",
"section":"waiting_period","doc_id":"H001"}),

Document(page_content=
"Cashless hospitalisation requires pre-authorisation from the TPA at least 48 "
"hours before planned admission. Emergency admissions must be intimated within "
"24 hours. Cashless available at 7000+ network hospitals.",
metadata={"policy_type":"health","claim_type":"cashless",
"section":"claims_procedure","doc_id":"H002"}),

Document(page_content=
"Maternity benefit covers normal delivery up to Rs 50,000 and Caesarean up to "
"Rs 75,000. Available after 2 continuous policy years. Newborn is covered from day 1.",
metadata={"policy_type":"health","claim_type":"maternity",
"section":"benefits","doc_id":"H003"}),

Document(page_content=
"No Claim Bonus in health insurance gives 5% increase in sum insured for each "
"claim-free year up to a maximum of 50% of the original sum insured.",
metadata={"policy_type":"health","claim_type":"bonus",
"section":"premium","doc_id":"H004"}),

Document(page_content=
"Day care procedures requiring less than 24 hours hospitalisation are covered. "
"Over 540 listed procedures are covered including cataract, chemotherapy, dialysis.",
metadata={"policy_type":"health","claim_type":"daycare",
```

```
    "section":"benefits","doc_id":"H005"}),
    Document(page_content=
    "Reimbursement claims must be submitted within 30 days of hospital discharge. "
    "Required: discharge summary, original bills, diagnostic reports, prescription, "
    "and completed claim form signed by treating doctor.",
    metadata={"policy_type":"health","claim_type":"reimbursement",
    "section":"claims_procedure","doc_id":"H006"}),
    Document(page_content=
    "Cosmetic surgery and dental treatment are excluded unless required due to an accident. "
    "Infertility treatments, IVF, and weight loss surgeries are also excluded.",
    metadata={"policy_type":"health","claim_type":"exclusion",
    "section":"exclusions","doc_id":"H007"}),
    Document(page_content=
    "Restoration benefit reinstates the full sum insured once per year after complete "
    "exhaustion, for unrelated illnesses only. Ensures family coverage is not lost.",
    metadata={"policy_type":"health","claim_type":"restoration",
    "section":"add_ons","doc_id":"H008"}),
    Document(page_content=
    "Air ambulance cover for life-threatening emergencies is available up to Rs 2,50,000 "
    "per policy year for inter-city air transfer to a higher-equipped hospital.",
    metadata={"policy_type":"health","claim_type":"emergency",
    "section":"add_ons","doc_id":"H009"}),
    Document(page_content=
    "Annual preventive health check-up is covered up to Rs 2,000 per adult member. "
    "Tests: CBC, lipid profile, blood sugar, ECG at empanelled diagnostic centres.",
    metadata={"policy_type":"health","claim_type":"wellness",
    "section":"benefits","doc_id":"H010"}),
    # ■■ Motor Insurance (M001-M010) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    Document(page_content=
    "Third party liability is mandatory under the Motor Vehicles Act. It covers death, "
    "bodily injury or property damage to a third party. Property damage capped Rs 7.5L.",
    metadata={"policy_type":"motor","claim_type":"third_party",
    "section":"coverage","doc_id":"M001"}),
    Document(page_content=
    "Own damage cover protects against accidental damage from collision, fire, theft, "
    "natural calamities, riots and malicious acts. Claim subject to IDV minus depreciation.",
    metadata={"policy_type":"motor","claim_type":"own_damage",
    "section":"coverage","doc_id":"M002"}),
    Document(page_content=
    "No Claim Bonus for motor insurance: 20% after 1 claim-free year, 25% after 2 years, "
    "35% after 3 years, 45% after 4 years, and 50% after 5+ consecutive claim-free years.",
    metadata={"policy_type":"motor","claim_type":"bonus",
    "section":"premium","doc_id":"M003"}),
    Document(page_content=
    "Cashless repair network covers 5000+ garages across India. Intimate insurer within "
    "24 hours of accident. Surveyor inspection is mandatory before vehicle is moved.",
    metadata={"policy_type":"motor","claim_type":"cashless",
    "section":"claims_procedure","doc_id":"M004"}),
```

```
    Document(page_content=
    "Zero depreciation add-on eliminates depreciation deductions on replaced parts. "
    "Available for vehicles up to 5 years old. Increases premium by 15-20%.",
    metadata={"policy_type":"motor","claim_type":"add_on",
    "section":"add_ons","doc_id":"M005"}),
    Document(page_content=
    "Engine protection add-on covers damage due to water ingression and hydrostatic lock. "
    "Essential for vehicles in flood-prone areas and during monsoon season in India.",
    metadata={"policy_type":"motor","claim_type":"flood",
    "section":"add_ons","doc_id":"M006"}),
    Document(page_content=
    "Driving under influence of alcohol or drugs voids the motor insurance claim. "
    "Other exclusions: no valid licence, commercial use of private vehicle, racing.",
    metadata={"policy_type":"motor","claim_type":"exclusion",
    "section":"exclusions","doc_id":"M007"}),
    Document(page_content=
    "Roadside assistance provides towing up to 50 km, battery jump-start, flat tyre "
    "change and emergency fuel delivery. Available 24x7 on highways and city roads.",
    metadata={"policy_type":"motor","claim_type":"add_on",
    "section":"add_ons","doc_id":"M008"}),
    Document(page_content=
    "Insured Declared Value is the current market value of the vehicle minus depreciation. "
    "IDV decreases 5-20% per year by age. It determines the maximum claim amount.",
    metadata={"policy_type":"motor","claim_type":"valuation",
    "section":"coverage","doc_id":"M009"}),
    Document(page_content=
    "Personal accident cover provides Rs 15 lakh to owner-driver for accidental death "
    "or permanent total disability. Mandatory under IRDAI regulations since 2019.",
    metadata={"policy_type":"motor","claim_type":"personal_accident",
    "section":"coverage","doc_id":"M010"}),
    # ■■ Life Insurance (L001-L010) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    Document(page_content=
    "Term life insurance provides pure death benefit with no maturity value. Sum assured "
    "is paid to nominee on insured death during policy term. Premiums are much lower.",
    metadata={"policy_type":"life","claim_type":"death_benefit",
    "section":"product_type","doc_id":"L001"}),
    Document(page_content=
    "Critical illness rider pays lump sum on diagnosis of cancer, heart attack, stroke, "
    "kidney failure or major organ transplant. Paid regardless of actual treatment cost.",
    metadata={"policy_type":"life","claim_type":"critical_illness",
    "section":"riders","doc_id":"L002"}),
    Document(page_content=
    "Suicide clause: death by suicide within the first year of policy is excluded. "
    "From year 2 onwards, 80% of total premiums paid are returned to the nominee.",
    metadata={"policy_type":"life","claim_type":"exclusion",
    "section":"exclusions","doc_id":"L003"}),
    Document(page_content=
    "Death claim process: nominee submits death certificate, original policy bond, "
```

```
    "NEFT details and ID proof within 90 days. Settled within 30 days if complete.",
    metadata={"policy_type":"life","claim_type":"death_claim",
    "section":"claims_procedure","doc_id":"L004"}),
    Document(page_content=
    "Free look period allows policy cancellation within 15 days of receiving the document. "
    "Full premium refunded after deducting proportionate risk premium and stamp duty.",
    metadata={"policy_type":"life","claim_type":"cancellation",
    "section":"policy_terms","doc_id":"L005"}),
    Document(page_content=
    "Accidental death benefit rider doubles the sum assured if death is accidental. "
    "Also covers permanent total disability with 100% of sum assured. Age 18-65.",
    metadata={"policy_type":"life","claim_type":"accidental_death",
    "section":"riders","doc_id":"L006"}),
    Document(page_content=
    "Premium waiver rider waives future premiums if policyholder becomes permanently "
    "disabled or critically ill. Policy continues in force with all benefits intact.",
    metadata={"policy_type":"life","claim_type":"disability",
    "section":"riders","doc_id":"L007"}),
    Document(page_content=
    "Grace period for life insurance is 30 days for monthly premium mode and 15 days "
    "for other frequencies. Policy lapses if not paid. Revival possible within 5 years.",
    metadata={"policy_type":"life","claim_type":"lapse",
    "section":"policy_terms","doc_id":"L008"}),
    Document(page_content=
    "Surrender value is available after 3 years of premiums paid. Guaranteed surrender "
    "value is 30% of premiums paid excluding first year. Special surrender value may be higher.",
    metadata={"policy_type":"life","claim_type":"surrender",
    "section":"policy_terms","doc_id":"L009"}),
    Document(page_content=
    "Nomination allows designating a nominee to receive the death benefit. "
    "Nominee can be changed at any time. Without nomination, legal heirs receive the amount.",
    metadata={"policy_type":"life","claim_type":"nomination",
    "section":"policy_terms","doc_id":"L010"}),
    ]
    if __name__ == "__main__":
    print(f"Corpus: {len(CORPUS)} documents")
    for pt in ["health","motor","life"]:
    n = sum(1 for d in CORPUS if d.metadata["policy_type"]==pt)
    print(f" {pt}: {n} docs")
```

Corpus Metadata Summary

Policy Type	Docs	Claim Types	IDs
Health	10	exclusion, cashless, maternity, bonus, daycare, reimbursement, restoration, emergency, wellness	H001-H010
Motor	10	third_party, own_damage, bonus, cashless, add_on, flood, exclusion, valuation, personal_accident	M001-M010

Life	10	death_benefit, critical_illness, exclusion, death_claim, cancellation, accidental_death, disability, lapse, surrender, nomination	L001-L010
Total	**30**	28 distinct claim_type values, 7 section categories	H001-L010

Lab 9 — FAISS Vector Store and Similarity Search

Build, persist and query an embedding index on the insurance corpus

Lab Objectives

- Understand FAISS flat L2 index and how k-NN retrieval works
- Embed all 30 corpus documents with OpenAI text-embedding-3-small
- Build a FAISS vector store using LangChain wrapper
- Run similarity_search_with_score and inspect distance values
- Persist index to disk and reload it (used by Labs 10-12)
- Execute 4 insurance test queries and verify top results

How FAISS Works

FAISS stores document vectors in a flat L2 index. At query time the query text is embedded into the same space and the k nearest neighbour vectors are returned with their L2 distances (lower = more similar). With 30 documents the search takes under 1 ms after embeddings are loaded.

Step 1 — Create Lab Folder

```
cd ~/insurance_ai_lab/lab9
touch faiss_search.py
code faiss_search.py
```

Step 2 — Imports, Embeddings and Corpus Load

```
# faiss_search.py
import os, sys, time, json
from dotenv import load_dotenv
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
sys.path.insert(0, "../shared")
from corpus import CORPUS
load_dotenv(dotenv_path="/root/insurance_ai_lab/.env")
embeddings = OpenAIEmbeddings(
model="text-embedding-3-small",
openai_api_key=os.getenv("OPENAI_API_KEY")
)
print(f"Corpus loaded: {len(CORPUS)} documents")
```

Step 3 — Build and Persist FAISS Index

```
def build_index(docs, save_path="faiss_insurance_index"):
print("Building FAISS index (embedding API calls)...")
t0 = time.time()
vs = FAISS.from_documents(docs, embeddings)
```

```
elapsed = round(time.time() - t0, 2)
vs.save_local(save_path)
print(f"Done in {elapsed}s -> saved to ./{save_path}/")
return vs, elapsed
def load_index(save_path="faiss_insurance_index"):
vs = FAISS.load_local(save_path, embeddings,
allow_dangerous_deserialization=True)
print(f"Index loaded from disk: {save_path}")
return vs
```

Step 4 — Similarity Search with Distance Scores

```
def search(vs, query, k=3):
t0 = time.time()
hits = vs.similarity_search_with_score(query, k=k)
lat = round((time.time() - t0) * 1000, 1)
print(f"\nQUERY: {query}")
print(f"Latency: {lat} ms | Top-{k} results:")
for rank, (doc, score) in enumerate(hits, 1):
m = doc.metadata
print(f" {rank}. [{m['doc_id']}] {m['policy_type']}/{m['claim_type']} score={score:.4f}")
print(f" {doc.page_content[:110]}...")
return hits, lat
```

Step 5 — Run 4 Insurance Test Queries

```
def main():
vs, _ = build_index(CORPUS)
queries = [
"How do I file a cashless hospital claim?",
"What is the no claim bonus for motor insurance?",
"Is my car engine covered if flooded during monsoon?",
"What documents are needed to claim life insurance after death?",
]
results = {}
for q in queries:
hits, lat = search(vs, q, k=3)
results[q] = {
"top_doc": hits[0][0].metadata["doc_id"],
"score": round(float(hits[0][1]), 4),
"latency_ms": lat
}
with open("lab9_results.json","w") as f:
json.dump(results, f, indent=2)
print("\nResults saved to lab9_results.json")
if __name__ == "__main__": main()
```

Step 6 — Run

```
python faiss_search.py
```

Expected Results

Query	Expected Top Doc	Why Relevant
"How to file cashless claim?"	H002	Directly describes cashless TPA pre-authorisation process
"Motor NCB percentage?"	M003	Contains exact 20%-25%-35%-45%-50% NCB ladder
"Engine flooded monsoon?"	M006	Engine protection add-on for water ingression stated explicitly
"Documents for life claim after death?"	L004	Death claim doc list: certificate, policy bond, NEFT, ID proof

Note FAISS returns L2 (Euclidean) distance — lower score = more similar. The saved index folder contains index.faiss and index.pkl. Labs 10-12 reload this same index instead of rebuilding.

Challenge Task

Add a 5th query: "What is the free look cancellation period for life insurance?" and verify doc L005 ranks in the top 3. Then try OpenAIEmbeddings(chunk_size=5) to control batch size during embedding.

Lab 10 — Semantic Search with Metadata Filters

Restrict FAISS search to policy_type, claim_type and section before ranking

Lab Objectives

- Understand why cross-product-type noise degrades answer quality
- Use the LangChain FAISS filter= argument for exact-match pre-filtering
- Filter by single field: policy_type = "motor"
- Filter by two fields: policy_type = "health" AND claim_type = "exclusion"
- Compare filtered vs unfiltered results side by side for the same query
- Build a reusable filtered_search() utility function

Why Metadata Filtering Matters

Without filters, a query like "Is cosmetic surgery covered?" may return motor and life exclusion documents alongside health exclusion documents. A customer asking about their health policy only needs health results. Metadata filters act as a pre-retrieval guard eliminating cross-category noise.

Without Filter	With Filter: policy_type = health
Top results: 1. H007 - health exclusion 2. M007 - motor exclusion 3. L003 - life exclusion	Top results (health only): 1. H007 - cosmetic exclusion 2. H001 - PED exclusion 3. H006 - reimbursement

Step 1 — Setup

```
cd ~/insurance_ai_lab/lab10
touch semantic_filter_search.py
code semantic_filter_search.py
```

Step 2 — Imports and Load Shared Index

```
# semantic_filter_search.py
import os, sys, time, json
from dotenv import load_dotenv
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
sys.path.insert(0, "../shared")
from corpus import CORPUS
load_dotenv(dotenv_path="/root/insurance_ai_lab/.env")
embeddings = OpenAIEmbeddings(model="text-embedding-3-small",
openai_api_key=os.getenv("OPENAI_API_KEY"))
# Reuse index built in Lab 9 — or rebuild if missing
IDX = "../lab9/faiss_insurance_index"
if os.path.exists(IDX):
vs = FAISS.load_local(IDX, embeddings,
allow_dangerous_deserialization=True)
print("Loaded index from Lab 9 cache")
```

```
else:
vs = FAISS.from_documents(CORPUS, embeddings)
vs.save_local(IDX)
print("Built and saved new index")
```

Step 3 — Filtered Search Function

```
def filtered_search(query, filters=None, k=3):
"""
Semantic search with optional metadata filter dict.
filters: e.g. {"policy_type": "health"}
or {"policy_type": "motor", "claim_type": "own_damage"}
"""
t0 = time.time()
if filters:
results = vs.similarity_search_with_score(query, k=k, filter=filters)
else:
results = vs.similarity_search_with_score(query, k=k)
lat = round((time.time() - t0) * 1000, 1)
tag = f"FILTERED({filters})" if filters else "UNFILTERED"
print(f"\n[{tag}]")
print(f"Q: {query} | {lat} ms | {len(results)} results")
for rank, (doc, score) in enumerate(results, 1):
m = doc.metadata
print(f" {rank}. [{m['doc_id']}] {m['policy_type']}/{m['claim_type']} score={score:.4f}")
print(f" {doc.page_content[:100]}...")
return results, lat
```

Step 4 — Run 6 Filter Scenarios

```
def main():
experiments = [
{"label": "Unfiltered baseline",
"query": "Is cosmetic surgery covered?",
"filter": None},
{"label": "Health-only filter",
"query": "Is cosmetic surgery covered?",
"filter": {"policy_type": "health"}},
{"label": "Motor claims_procedure",
"query": "How do I register a vehicle damage claim?",
"filter": {"policy_type": "motor", "section": "claims_procedure"}},
{"label": "Life riders only",
"query": "What extra coverage can I add to my life policy?",
"filter": {"policy_type": "life", "section": "riders"}},
{"label": "Motor add-ons monsoon",
"query": "What add-ons protect my car in monsoon floods?",
"filter": {"policy_type": "motor", "section": "add_ons"}},
{"label": "Health exclusion dual-filter",
```

```
    "query": "What is not covered under health insurance?",
    "filter": {"policy_type": "health", "claim_type": "exclusion"}},
    ]
    records = []
    for exp in experiments:
    res, lat = filtered_search(exp["query"], exp["filter"])
    records.append({
    "label": exp["label"],
    "filter": str(exp["filter"]),
    "latency_ms": lat,
    "results": len(res)
    })
    with open("lab10_results.json","w") as f:
    json.dump(records, f, indent=2)
    if __name__ == "__main__": main()
```

```
python semantic_filter_search.py
```

Observation Table

Scenario	Filter	Expected Top-1	Quality Gain
Unfiltered cosmetic	None	H007 or M007?	Mixed policy types confuse user
Health-only cosmetic	policy_type=health	H007	100% health docs; direct exclusion answer
Motor claims_procedure	motor + section=claims_procedure	M004	Garage process isolated from NCB/OD docs
Life riders	life + section=riders	L002 or L006	Only critical illness, ADB, waiver docs
Motor add-ons monsoon	motor + section=add_ons	M006	Engine protection clause surfaces first
Health exclusion dual	health + claim=exclusion	H007 or H001	Both health exclusion docs returned, no motor/life noise

Challenge Task

Build a smart_search() wrapper that auto-infers the filter from the query text: if "motor" or "car" appears -> filter policy_type=motor; if "life" or "nominee" appears -> filter life; else unfiltered. Test: "When does my car NCB reset?" vs "When does NCB reset?"

Lab 11 — Hybrid Search: BM25 + FAISS + RRF Fusion

Combine keyword ranking with embedding ranking using Reciprocal Rank Fusion

Lab Objectives

- Understand when keyword search wins vs when semantic search wins
- Build a BM25Okapi index over the 30-document corpus
- Score documents independently with BM25 and FAISS
- Fuse both ranked lists using Reciprocal Rank Fusion (RRF, k=60)
- Run 4 queries designed to highlight each engine strength
- Print a three-column comparison matrix per query

When Each Engine Wins

BM25 Wins When...	FAISS Wins When...	Hybrid Wins When...
- Query uses exact insurance terms - "IDV", "IRDAI", "Rs 7.5L" - Regulatory jargon - Clause reference by ID	- Query uses everyday language - "Flood damage to my car" - "Can I cancel and get refund?" - Paraphrased concepts	- Mixed queries: acronym + concept - "NCB rules for accident car" - "What if I die and family suffers?" - Production RAG systems

Step 1 — Setup

```
cd ~/insurance_ai_lab/lab11
pip install rank-bm25 --quiet
touch hybrid_search.py
code hybrid_search.py
```

Step 2 — Imports and Initialise Both Engines

```
# hybrid_search.py
import os, sys, time, json
import numpy as np
from rank_bm25 import BM25Okapi
from dotenv import load_dotenv
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
sys.path.insert(0, "../shared")
from corpus import CORPUS
load_dotenv(dotenv_path="/root/insurance_ai_lab/.env")
embeddings = OpenAIEmbeddings(model="text-embedding-3-small",
openai_api_key=os.getenv("OPENAI_API_KEY"))
# Build BM25 index
texts = [d.page_content for d in CORPUS]
tokenized = [t.lower().split() for t in texts]
bm25 = BM25Okapi(tokenized)
print("BM25 index built")
# Load or build FAISS index
```

```
IDX = "../lab9/faiss_insurance_index"
if os.path.exists(IDX):
faiss_vs = FAISS.load_local(IDX, embeddings,
allow_dangerous_deserialization=True)
else:
faiss_vs = FAISS.from_documents(CORPUS, embeddings)
faiss_vs.save_local(IDX)
```

Step 3 — BM25 Search Function

```
def bm25_search(query, k=6):
scores = bm25.get_scores(query.lower().split())
top_idx = np.argsort(scores)[::-1][:k]
return [(CORPUS[i], float(scores[i])) for i in top_idx]
```

Step 4 — Reciprocal Rank Fusion

RRF formula: for each document in each ranked list, add 1/(k+rank) to its score. k=60 is the standard constant that dampens rank influence. Documents appearing in both lists receive the highest combined scores.

```
def rrf_fuse(list1, list2, k=60):
scores = {}
docs = {}
for rank, (doc, _) in enumerate(list1, 1):
did = doc.metadata["doc_id"]
scores[did] = scores.get(did, 0) + 1.0 / (k + rank)
docs[did] = doc
for rank, (doc, _) in enumerate(list2, 1):
did = doc.metadata["doc_id"]
scores[did] = scores.get(did, 0) + 1.0 / (k + rank)
docs[did] = doc
ranked = sorted(scores.keys(), key=lambda x: scores[x], reverse=True)
return [(docs[d], scores[d]) for d in ranked]
```

Step 5 — Hybrid Search with 3-Column Output

```
def hybrid_search(query, k=3):
t0 = time.time()
b_res = bm25_search(query, k=k*2)
f_res = faiss_vs.similarity_search_with_score(query, k=k*2)
h_res = rrf_fuse(b_res, f_res)[:k]
lat = round((time.time()-t0)*1000, 1)
print(f"\nQUERY: {query} [{lat} ms]")
print(f" RANK BM25 FAISS HYBRID")
print(f" ---- -------------- -------------- --------------")
for i in range(k):
b = b_res[i][0].metadata["doc_id"] if i<len(b_res) else "-"
f = f_res[i][0].metadata["doc_id"] if i<len(f_res) else "-"
```

```
    h = h_res[i][0].metadata["doc_id"] if i<len(h_res) else "-"
    print(f" {i+1} {b:<14} {f:<14} {h:<14}")
    return {"bm25":b_res[:k],"faiss":f_res[:k],"hybrid":h_res,"lat":lat}
```

Step 6 — Run 4 Targeted Queries and Save Results

```
def main():
queries = [
("IDV insured declared value depreciation", "M009",
"BM25 wins — exact insurance acronym"),
("My car got flooded in the rains am I covered", "M006",
"FAISS wins — paraphrased conceptual query"),
("NCB for accident free car policy", "M003",
"Hybrid wins — keyword + concept mix"),
("what happens to my family if I pass away", "L001",
"FAISS wins — everyday language for death benefit"),
]
records = []
for query, expected, note in queries:
res = hybrid_search(query, k=3)
print(f" Expected: {expected} | {note}")
records.append({
"query": query, "expected": expected,
"bm25_top1": res["bm25"][0][0].metadata["doc_id"],
"faiss_top1": res["faiss"][0][0].metadata["doc_id"],
"hybrid_top1": res["hybrid"][0][0].metadata["doc_id"],
"lat_ms": res["lat"]
})
with open("lab11_results.json","w") as f:
json.dump(records, f, indent=2)
if __name__ == "__main__": main()
```

```
python hybrid_search.py
```

Expected Comparison Matrix

Query Type	Expected	BM25	FAISS	Hybrid	Winner
Acronym: "IDV"	M009	M009	M002?	M009	BM25
Paraphrase: "car flooded rains"	M006	M002?	M006	M006	FAISS
Mixed: "NCB accident free car"	M003	M003	M003	M003	All equal
Everyday: "pass away family"	L001	L003?	L001	L001	FAISS/Hybrid

Challenge Task

Combine metadata-aware hybrid search: before running FAISS, detect the policy type from the query and apply a filter. Fuse only the filtered FAISS results with BM25. Does this improve precision for the "NCB accident free car" query?

Lab 12 — Retrieval Benchmarking

Measure Latency, Precision@3 and Recall@3 across BM25, FAISS and Hybrid

Lab Objectives

- Define an 8-query ground-truth evaluation set with known relevant doc IDs
- Measure wall-clock latency for each engine per query
- Compute Precision@3 and Recall@3 for each engine on each query
- Print a complete benchmark matrix with per-query and aggregate scores
- Identify best engine per query category (process, term, conversational, technical)
- Produce a final recommendation for the insurance RAG pipeline

Evaluation Metrics

Metric	Formula	What It Measures
Precision@K	Relevant docs in top-K / K	What fraction of retrieved docs are actually relevant? Measures accuracy.
Recall@K	Relevant docs in top-K / Total relevant docs	What fraction of all relevant docs did we find? Measures coverage.
Latency (ms)	time.time() delta x 1000	Wall-clock time from query to ranked results. Lower is better.
F1@K	2 x P@K x R@K / (P@K + R@K)	Harmonic mean of Precision and Recall. Balanced single metric.

Step 1 — Setup

```
cd ~/insurance_ai_lab/lab12
touch benchmark.py
code benchmark.py
```

Step 2 — Imports, Engines and Ground Truth Dataset

```
# benchmark.py
import os, sys, time, json
import numpy as np
from rank_bm25 import BM25Okapi
from dotenv import load_dotenv
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
sys.path.insert(0, "../shared")
from corpus import CORPUS
load_dotenv(dotenv_path="/root/insurance_ai_lab/.env")
embeddings = OpenAIEmbeddings(model="text-embedding-3-small",
openai_api_key=os.getenv("OPENAI_API_KEY"))
# Ground truth: query + all relevant doc IDs
EVAL_SET = [
```

```
{"id":"Q1","query":"how to file cashless hospitalisation claim",
"relevant":["H002","H006"],"category":"process"},
{"id":"Q2","query":"motor NCB no claim bonus percentage",
"relevant":["M003"],"category":"term"},
{"id":"Q3","query":"car engine damage flood water ingression monsoon",
"relevant":["M006"],"category":"technical"},
{"id":"Q4","query":"what happens to insurance if I die",
"relevant":["L001","L004"],"category":"conversational"},
{"id":"Q5","query":"pre-existing disease waiting period 36 months",
"relevant":["H001"],"category":"term"},
{"id":"Q6","query":"cancellation free look period life policy refund",
"relevant":["L005"],"category":"process"},
{"id":"Q7","query":"disability premium waiver benefit",
"relevant":["L007"],"category":"term"},
{"id":"Q8","query":"zero depreciation add on new car protection",
"relevant":["M005"],"category":"recommendation"},
]
```

Step 3 — Initialise Engines and Scoring Functions

```
# BM25
texts = [d.page_content for d in CORPUS]
bm25 = BM25Okapi([t.lower().split() for t in texts])
# FAISS
IDX = "../lab9/faiss_insurance_index"
faiss_vs = (FAISS.load_local(IDX, embeddings, allow_dangerous_deserialization=True)
if os.path.exists(IDX)
else FAISS.from_documents(CORPUS, embeddings))
def rrf(r1, r2, k=60):
sc, docs = {}, {}
for rank,(doc,_) in enumerate(r1,1):
did=doc.metadata["doc_id"]; sc[did]=sc.get(did,0)+1/(k+rank); docs[did]=doc
for rank,(doc,_) in enumerate(r2,1):
did=doc.metadata["doc_id"]; sc[did]=sc.get(did,0)+1/(k+rank); docs[did]=doc
return [(docs[d],sc[d]) for d in sorted(sc,key=lambda x:sc[x],reverse=True)]
def precision_k(retrieved, relevant, k): return round(sum(1 for d in retrieved[:k] if d in
relevant)/k, 3)
def recall_k(retrieved, relevant, k): return round(sum(1 for d in retrieved[:k] if d in
relevant)/len(relevant),3) if relevant else 0
```

Step 4 — Benchmark Runner

```
def run_benchmark(k=3):
records = []
for q in EVAL_SET:
query = q["query"]
relevant = set(q["relevant"])
# BM25
t0 = time.time()
```

```
    raw = bm25.get_scores(query.lower().split())
    bm_ids = [CORPUS[i].metadata["doc_id"] for i in np.argsort(raw)[::-1][:k*2]]
    bm_lat = round((time.time()-t0)*1000,1)
    bm_tup = [(CORPUS[i],float(raw[i])) for i in np.argsort(raw)[::-1][:k*2]]
    # FAISS
    t0 = time.time()
    fa_res = faiss_vs.similarity_search_with_score(query, k=k*2)
    fa_ids = [d.metadata["doc_id"] for d,_ in fa_res]
    fa_lat = round((time.time()-t0)*1000,1)
    # Hybrid
    t0 = time.time()
    hy_res = rrf(bm_tup, fa_res)
    hy_ids = [d.metadata["doc_id"] for d,_ in hy_res]
    hy_lat = round((time.time()-t0)*1000,1)
    records.append({
    "id":q["id"], "query":query[:38], "cat":q["category"],
    "bm_p":precision_k(bm_ids,relevant,k), "bm_r":recall_k(bm_ids,relevant,k), "bm_lat":bm_lat,
    "fa_p":precision_k(fa_ids,relevant,k), "fa_r":recall_k(fa_ids,relevant,k), "fa_lat":fa_lat,
    "hy_p":precision_k(hy_ids,relevant,k), "hy_r":recall_k(hy_ids,relevant,k), "hy_lat":hy_lat,
    })
    return records
```

Step 5 — Print Matrix and Aggregate Averages

```
    def print_results(records):
    print("="*88)
    print("BENCHMARK — Insurance Retrieval (K=3)")
    print("="*88)
    print(f" ID Query BM25 P/R FAISS P/R Hybrid P/R")
    print("-"*88)
    for r in records:
    bpr = f"{r['bm_p']:.2f}/{r['bm_r']:.2f}"
    fpr = f"{r['fa_p']:.2f}/{r['fa_r']:.2f}"
    hpr = f"{r['hy_p']:.2f}/{r['hy_r']:.2f}"
    print(f" {r['id']:<4} {r['query']:<35} {bpr:^12} {fpr:^12} {hpr:^12}")
    print("="*88)
    for eng,pk,rk,lk in [("BM25","bm_p","bm_r","bm_lat"),
    ("FAISS","fa_p","fa_r","fa_lat"),
    ("Hybrid","hy_p","hy_r","hy_lat")]:
    avgp=sum(r[pk] for r in records)/len(records)
    avgr=sum(r[rk] for r in records)/len(records)
    avgl=sum(r[lk] for r in records)/len(records)
    print(f" {eng:<8} Avg P@3={avgp:.3f} Avg R@3={avgr:.3f} Avg Lat={avgl:.1f}ms")
    def main():
    records = run_benchmark(k=3)
    print_results(records)
    with open("lab12_benchmark.json","w") as f:
    json.dump(records, f, indent=2)
```

```
if __name__ == "__main__": main()
```

```
python benchmark.py
```

Expected Final Matrix

Query	Cat	BM25 P/R	FAISS P/R	Hybrid P/R	Best
cashless hospitalisation	process	0.33/0.50	0.67/1.00	0.67/1.00	FAISS/HYB
motor NCB percentage	term	1.00/1.00	1.00/1.00	1.00/1.00	All equal
engine flood monsoon	technical	0.33/1.00	1.00/1.00	1.00/1.00	FAISS/HYB
die insurance family	conversational	0.33/0.50	0.67/1.00	0.67/1.00	FAISS/HYB
pre-existing 36 months	term	1.00/1.00	1.00/1.00	1.00/1.00	All equal
free look cancellation	process	0.33/1.00	1.00/1.00	1.00/1.00	FAISS/HYB
disability premium waiver	term	0.67/1.00	1.00/1.00	1.00/1.00	FAISS/HYB
zero depreciation car	recom.	0.33/1.00	1.00/1.00	1.00/1.00	FAISS/HYB
AVERAGE	—	**0.54/0.88**	**0.92/1.00**	**0.92/1.00**	**Hybrid**

Key Takeaway

BM25 alone achieves ~54% P@3 because it misses paraphrased queries. FAISS semantic search achieves ~92% P@3. Hybrid matches FAISS precision while also handling exact-term queries — making it the safest default for production insurance RAG.

Challenge Task

Add a 4th filtered_faiss engine that applies an auto-inferred policy_type filter before FAISS retrieval. Add it as a new column in the benchmark matrix. Does filtering hurt recall for the conversational query Q4?

Troubleshooting, Commands and Glossary

Errors with fixes, quick command reference, 13-term glossary

Error: ImportError: No module named faiss

Cause: faiss-cpu not installed.

```
Fix: pip install faiss-cpu
```

Error: ImportError: No module named rank_bm25

Cause: rank-bm25 package not installed.

```
Fix: pip install rank-bm25
```

Error: FAISS allow_dangerous_deserialization must be True

Cause: Loading pickled index requires explicit opt-in for security.

```
Fix: Add allow_dangerous_deserialization=True to FAISS.load_local()
```

Error: Filter returns 0 results

Cause: Filter value case mismatch (e.g. 'Health' vs 'health').

```
Fix: Print doc.metadata for 2-3 docs and copy exact value strings.
```

Error: BM25 scores all 0.0

Cause: Query tokens not present in any document — out of vocabulary.

```
Fix: Try shorter queries. BM25 needs exact token overlap with documents.
```

Error: FAISS index inconsistent after adding docs

Cause: Flat index cannot be updated incrementally — must be rebuilt.

```
Fix: Call FAISS.from_documents(all_docs, embeddings) and save again.
```

Error: FAISS latency 200ms or more

Cause: Embedding API call dominates, not FAISS search (which is under 1ms).

```
Fix: Pre-embed queries or use a local embedding model for production.
```

Quick Command Reference

Action	Command
Verify corpus	`cd ~/insurance_ai_lab && python shared/corpus.py`
Run Lab 9 (FAISS)	`cd ~/insurance_ai_lab/lab9 && python faiss_search.py`
Run Lab 10 (filters)	`cd ~/insurance_ai_lab/lab10 && python semantic_filter_search.py`

Run Lab 11 (hybrid)	`cd ~/insurance_ai_lab/lab11 && python hybrid_search.py`
Run Lab 12 (benchmark)	`cd ~/insurance_ai_lab/lab12 && python benchmark.py`
Inspect FAISS index files	`ls -lh lab9/faiss_insurance_index/`
View benchmark results	`cat lab12/lab12_benchmark.json | python3 -m json.tool`
Check GPU for FAISS	`python3 -c "import faiss; print(faiss.get_num_gpus())"`

Key Concepts Glossary

Term	Definition
FAISS	Facebook AI Similarity Search — in-memory approximate nearest-neighbour index
k-NN Search	k Nearest Neighbours — find k vectors closest to the query vector
L2 Distance	Euclidean distance; lower value = more similar in FAISS
BM25 / BM25Okapi	Probabilistic keyword ranking using TF, IDF and document length
RRF	Reciprocal Rank Fusion — combines ranked lists: score = sum of 1/(k+rank)
Metadata Filter	Pre-retrieval exact-match constraint: restrict to field=value before ranking
Precision@K	Fraction of top-K retrieved docs that are relevant. Accuracy metric.
Recall@K	Fraction of all relevant docs found in top-K. Coverage metric.
F1@K	Harmonic mean of Precision@K and Recall@K. Balanced metric.
text-embedding-3-small	OpenAI embedding model: 1536 dims, 8K token limit, cost-efficient
IDV	Insured Declared Value — vehicle market value minus depreciation; basis for motor claims
TPA	Third Party Administrator — handles cashless claim authorisation for health insurance
NCB	No Claim Bonus — reward for claim-free years: discount on motor OD or SI increase in health

Module 3 Complete! You have built a full insurance retrieval pipeline: FAISS vector similarity search, metadata-filtered semantic search, BM25 + FAISS hybrid with RRF fusion, and a rigorous benchmarking framework with Precision@3, Recall@3 and latency measurement. These are the core techniques behind enterprise RAG systems in insurance, banking and legal domains.